

Fase 1: Backend Core

AWS SAM + Lambda + DynamoDB Multi-Tenant

Tutorial paso a paso

Proyecto Sentinel AcademIA
lFunkown

20 de junio de 2026

¿Qué construimos en este tutorial?

Este documento es el log completo de la **Fase 1** del proyecto Sentinel AcademIA: el backend serverless en AWS Academy con Lambdas Python, DynamoDB multi-tenant, y API Gateway.

Stack tecnológico:

- Python 3.12 (runtime de Lambda) + 3.14.3 (local)
- AWS SAM (Infrastructure as Code)
- Lambda + API Gateway REST + DynamoDB
- Pydantic v2 para validación
- aws-lambda-powertools para observabilidad
- OpenAPI spec como contrato

Lo que verás:

- Estructura de carpetas y archivos
- Cada módulo compartido con su propósito
- El template SAM con todas las decisiones explicadas
- Las trampas de AWS Academy (y cómo resolverlas)
- 5 smoke tests que validan todo end-to-end

Aviso: AWS Academy es restrictivo

Este proyecto corre en **AWS Academy Learner Lab**, que tiene límites que no aplican a AWS normal. En particular, **no podemos crear IAM roles**. Esto afecta TODO el flujo y se explica en detalle en la sección correspondiente.

Índice

1. Prerrequisitos	3
1.1. Herramientas instaladas	3
1.1.1. Instala Python 3.12 con mise (si no lo tienes)	3
1.2. Credenciales de AWS Academy	3
2. Estructura del proyecto	3
2.1. Decisiones clave de la estructura	4

3. Configuración inicial: pyproject.toml	4
3.1. requirements.txt para SAM	5
4. Los 8 módulos compartidos (__shared/)	5
4.1. config.py — Configuración desde env vars	5
4.2. logger.py — Logger estructurado con powertools	5
4.3. errors.py — Excepciones tipadas + error handler	6
4.4. http_response.py — Respuestas estandar	7
4.5. correlation_id.py — Trazabilidad end-to-end	8
4.6. dynamo_client.py — Wrapper multi-tenant	8
4.7. schemas.py — Validación con Pydantic	9
4.8. auth.py — Multi-tenant	10
5. Las 4 Lambdas	10
5.1. health_check.py — Endpoint sin auth	10
5.2. create_queja.py — POST 202 Accepted (async pattern)	10
5.3. get_queja.py y list_quejas.py	12
6. SAM template — Infraestructura como código	12
6.1. El template completo (infra/template.yaml)	12
7. AWS Academy: las trampas y cómo resolverlas	14
7.1. Regla de oro: SIEMPRE LabRole	14
7.2. No usar Policies: en las funciones	14
7.3. CodeUri: ../src (no ../ ni ../../)	14
7.4. El bug No module named 'src'	15
8. Build, deploy, y samconfig.toml	15
8.1. samconfig.toml — Configuración persistente	15
8.2. El ciclo build → deploy	15
9. Los 5 smoke tests que validan todo	16
9.1. Output esperado de cada test	16
10. Catálogo de errores y soluciones	17
11. Lo que aprendimos (resumen ejecutivo)	18
12. Ejercicios para practicar	18
12.1. Ejercicio 1: Añadir un endpoint GET /api/quejas/{id}/status	18
12.2. Ejercicio 2: Añadir filtro por status en list_quejas	18
12.3. Ejercicio 3: Añadir paginación con cursor real	19
12.4. Ejercicio 4: Crear script de smoke tests	19
12.5. Ejercicio 5: Deploy con sam sync	19
13. Siguiendo el paso: Fase 2	19

1. Prerrequisitos

1.1. Herramientas instaladas

Herramienta	Versión	Check
AWS CLI	2.35+	<code>aws --version</code>
SAM CLI	1.162+	<code>sam --version</code>
Python 3.12	3.12.13	<code>python3.12 --version</code>
Python 3.14.3	3.14.3	<code>python3 --version</code>
mise	2026.6+	<code>mise --version</code>
Docker	29+	<code>docker --version</code>

1.1.1. Instala Python 3.12 con mise (si no lo tienes)

AWS Lambda runtime es Python 3.12. Tu sistema puede tener 3.14 (vía OCI installer). Necesitas 3.12 también:

```
1 mise use --global python@3.12
2 mise exec -- python3.12 --version # debe decir 3.12.13
```

1.2. Credenciales de AWS Academy

Las credenciales **no** se guardan en archivos del proyecto. Viven solo en variables de entorno del shell:

```
1 export AWS_ACCESS_KEY_ID="ASIAT..."
2 export AWS_SECRET_ACCESS_KEY="..."
3 export AWS_SESSION_TOKEN="IQoJb3JpZ2luX2Vj..."
4 export AWS_DEFAULT_REGION="us-east-1"
5 export AWS_REGION="us-east-1"
6
7 aws sts get-caller-identity
8 # debe mostrar: Account "227165337884" o el de tu lab
```

Vida útil: 3-4 horas. Cuando expiren, repites el flujo.

2. Estructura del proyecto

```
1 sentinel-academia/
2 |-- infra/
3 |   |-- template.yaml           # SAM template (IaC)
4 |   '-- samconfig.toml         # Configuración de sam deploy
5 |
6 |-- src/
7 |   |-- handlers/               # 1 archivo = 1 Lambda
8 |   |   |-- _shared/            # Módulos compartidos entre Lambdas
9 |   |   |   |-- __init__.py     # <-- IMPORTANTE: marca como package
10 |   |   |   |-- config.py
11 |   |   |   |-- logger.py
12 |   |   |   |-- errors.py
13 |   |   |   |-- http_response.py
14 |   |   |   |-- correlation_id.py
15 |   |   |   |-- dynamo_client.py
16 |   |   |   |-- schemas.py
17 |   |   '-- auth.py
```

```

18 | | | -- __init__.py          # <-- IMPORTANTE: marca como package
19 | | | -- health_check.py
20 | | | -- create_queja.py
21 | | | -- get_queja.py
22 | | | -- list_quejas.py
23 | |
24 | | -- requirements.txt      # Runtime deps (las usa SAM)
25 |
26 | -- pyproject.toml          # Configuración completa (dev + runtime)
27 | -- .opencode/
28 | | -- rules/10-aws-academy.md # Regla específica de Academy
29 | -- api-mock/openapi.yaml   # Spec de la API

```

2.1. Decisiones clave de la estructura

- **✓ 1 archivo = 1 Lambda:** cada endpoint vive en su propio archivo. Fácil de testear, deployar, y razonar.
- **✓ __shared/ para código común:** 8 módulos que usan TODAS las Lambdas.
- **✓ infra/ separado de src/:** el template vive aparte del código, evita ciclos de importación.
- **✓ __init__.py en cada package:** sin esto, Python no reconoce la carpeta como package.
- **✗ NO usamos src/handlers/ como prefijo:** los handlers son top-level para que Lambda los importe fácil.

3. Configuración inicial: pyproject.toml

El archivo `pyproject.toml` centraliza la configuración del proyecto Python: dependencias, herramientas de calidad, y versionado.

```

1 [project]
2 name = "sentinel-academia"
3 version = "0.1.0"
4 requires-python = ">=3.12,<3.15" # Lambda = 3.12, local = 3.14
5 dependencies = [
6     "aws-lambda-powertools[tracer,logger,metrics]>=3.0.0",
7     "boto3>=1.40.0",
8     "boto3-stubs[dynamodb,s3,ses,sns,sqs]>=1.40.0",
9     "pydantic>=2.8.0",
10    "pydantic-settings>=2.4.0",
11    "orjson>=3.10.0",
12 ]
13
14 [project.optional-dependencies]
15 dev = [
16     "pytest>=8.3.0",
17     "moto[dynamodb,s3,ses,sns,sqs,secretsmanager]>=5.0.0",
18     "ruff>=0.6.0",
19     "mypy>=1.11.0",
20 ]

```

Listing 1: `pyproject.toml` (deps runtime)

3.1. requirements.txt para SAM

AWS SAM NO lee pyproject.toml para dependencias. Necesita un requirements.txt plano:

```

1 # Runtime dependencies for Sentinel AcademIA Lambdas
2 aws-lambda-powertools[tracer,logger,metrics]>=3.0.0
3 boto3>=1.40.0
4 boto3-stubs[dynamodb]>=1.40.0
5 pydantic>=2.8.0
6 pydantic-settings>=2.4.0
7 orjson>=3.10.0
8 email-validator>=2.2.0 # <-- IMPORTANTE para EmailStr de Pydantic

```

Listing 2: src/requirements.txt

Trampa #1: email-validator

Si usas EmailStr de Pydantic, necesitas email-validator. Si no, en runtime:

```

1 Runtime.ImportModuleError: email-validator is not installed,
2   run 'pip install 'pydantic[email]''

```

4. Los 8 módulos compartidos (__shared/)

4.1. config.py — Configuración desde env vars

```

1 from functools import lru_cache
2 from pydantic_settings import BaseSettings
3
4 class Settings(BaseSettings):
5     app_name: str = "sentinel-academia"
6     stage: str = "dev"
7     region: str = "us-east-1"
8     dynamodb_table: str = "sentinel-quejas-{stage}"
9     files_bucket: str = "sentinel-files-{stage}"
10    log_level: str = "INFO"
11    allowed_origins: str = "*"
12    # ... OCI vars (Fase 2)
13
14 @lru_cache(maxsize=1)
15 def get_settings() -> Settings:
16     return Settings() # lee de env vars automáticamente

```

Listing 3: handlers/__shared/config.py (resumen)

Conceptos clave:

- ✓ Pydantic BaseSettings lee env vars **automáticamente**
- ✓ @lru_cache garantiza un singleton por invocación Lambda
- ✓ stage permite tener dev/staging/prod en la misma cuenta
- ✗ Nunca hardcodees valores; siempre desde env vars

4.2. logger.py — Logger estructurado con powertools

```

1 from aws_lambda_powertools import Logger
2
3 _loggers: dict[str, Logger] = {}
4
5 def get_logger(name: str) -> Logger:
6     if name in _loggers:
7         return _loggers[name]
8     settings = get_settings()
9     logger = Logger(
10         service=settings.app_name,
11         level=settings.log_level,
12     )
13     _loggers[name] = logger
14     return logger

```

Listing 4: handlers/_shared/logger.py

Reglas:

- **✗NUNCA** uses `print()` — siempre `logger.info()`
- **✓**El logger incluye automáticamente `correlation_id` y `tenant_id`
- **✓**Niveles: DEBUG, INFO, WARNING, ERROR

4.3. errors.py — Excepciones tipadas + error handler

```

1 class AppError(Exception):
2     def __init__(self, message, status_code=500,
3                 error_code="INTERNAL_ERROR",
4                 details=None):
5         super().__init__(message)
6         self.message = message
7         self.status_code = status_code
8         self.error_code = error_code
9         self.details = details or {}
10
11 class ValidationError(AppError): # 400
12     def __init__(self, message, details=None):
13         super().__init__(message, 400, "VALIDATION_ERROR", details)
14
15 class UnauthorizedError(AppError): # 401
16     def __init__(self, message="Unauthorized"):
17         super().__init__(message, 401, "UNAUTHORIZED")
18
19 class NotFoundError(AppError): # 404
20     ...
21
22 def error_handler(func):
23     def wrapper(event, context):
24         try:
25             return func(event, context)
26         except AppError as e:
27             log.warning("AppError raised", extra={
28                 "error_code": e.error_code,
29                 "status_code": e.status_code,
30                 "error_message": e.message, # <-- no "message"!

```

```

31         return build_error_response(
32             status_code=e.status_code,
33             error_code=e.error_code,
34             message=e.message,
35             details=e.details,
36         )
37     except Exception as e:
38         log.exception("Unhandled exception")
39         return build_error_response(500, "INTERNAL_ERROR", ...)
40     return wrapper

```

Listing 5: handlers/_shared/errors.py (nucleo)

Trampa #2: KeyError 'message' en logger

El módulo logging de Python tiene un atributo interno `message` en `LogRecord`. Si pasas `extra={"message": ...}` al logger, falla con `KeyError: "Attempt to overwrite 'message' in LogRecord"`.

Solución: usa `error_message` o cualquier otro nombre.

4.4. http_response.py — Respuestas estandar

```

1 class HttpResponse:
2     @staticmethod
3     def ok(body): return build_response(200, body)
4     @staticmethod
5     def created(body): return build_response(201, body)
6     @staticmethod
7     def accepted(body): return build_response(202, body) # <-- async
8     @staticmethod
9     def bad_request(msg, details=None):
10         return build_error_response(400, "VALIDATION_ERROR", msg,
11                                     details)
12         # ... not_found, conflict, internal_error, etc.
13
14 def build_error_response(status_code, error_code, message,
15                          details=None):
16     body = {
17         "code": error_code,
18         "message": message,
19         "correlationId": get_correlation_id() or "",
20         "timestamp": datetime.now(timezone.utc).isoformat(),
21     }
22     if details:
23         body["details"] = details
24     return build_response(status_code, body)

```

Listing 6: handlers/_shared/http_response.py (fragmento)

Decisiones:

- **✓CORS siempre:** headers en TODAS las respuestas
- **✓orjson** para serializar (más rápido que `stdlib`)
- **✓Formato de error alineado con OpenAPI spec:** `{code, message, correlationId, timestamp}`

4.5. correlation_id.py — Trazabilidad end-to-end

```

1 import uuid
2 from contextvars import ContextVar
3
4 _correlation_id_var: ContextVar[str | None] = ContextVar(
5     "correlation_id", default=None
6 )
7
8 def with_correlation_id(event):
9     headers = event.get("headers", {}) or {}
10    cid = (headers.get("x-correlation-id")
11           or headers.get("X-Correlation-ID")
12           or headers.get("X-Correlation-Id"))
13    if not cid:
14        cid = str(uuid.uuid4())
15    _correlation_id_var.set(cid)
16    return cid
17
18 def get_correlation_id() -> str | None:
19    return _correlation_id_var.get()

```

Listing 7: handlers/_shared/correlation_id.py

Conceptos:

- ✓ ContextVar: variable local al contexto de ejecución (no global)
- ✓ Header X-Correlation-ID opcional; se genera UUID si falta
- ✓ Aparece en logs y en respuestas (header X-Correlation-ID)

4.6. dynamo_client.py — Wrapper multi-tenant

```

1 def tenant_pk(tenant_id: str) -> str:
2     return f"TENANT#{tenant_id}"
3
4 def queja_pk(tenant_id: str, queja_id: str) -> str:
5     return f"{tenant_pk(tenant_id)}#QUEJA#{queja_id}"
6
7 def queja_sk(queja_id: str) -> str:
8     return f"QUEJA#{queja_id}"
9
10 class DynamoClient:
11     def put_item(self, item, condition_expression=None):
12         # PK/SK ya vienen en el item
13         kwargs = {"Item": item}
14         if condition_expression:
15             kwargs["ConditionExpression"] = condition_expression
16         self._table.put_item(**kwargs)
17
18     def get_queja(self, tenant_id, queja_id):
19         item = self.get_item(
20             pk=queja_pk(tenant_id, queja_id),
21             sk=queja_sk(queja_id)
22         )
23         if item is None:
24             raise NotFoundError(...)
25         return item
26

```



```

27     def query_by_tenant(self, tenant_id, sk_prefix=None, limit=None):
28         kc = Key("pk").eq(tenant_pk(tenant_id))
29         if sk_prefix:
30             kc = kc & Key("sk").begins_with(sk_prefix)
31         return self._table.query(KeyConditionExpression=kc,
                                   ...)["Items"]

```

Listing 8: handlers/_shared/dynamo_client.py (key builders)

Patrones clave:

- **✓ Single-table:** PK = TENANT#xxx#QUEJA#yyy, SK = QUEJA#yyy
- **✓ Multi-tenant safety:** las queries SIEMPRE filtran por tenant
- **✓ GSI para queries globales:** gsi1pk = TENANT#xxx#STATUS#PENDIENTE

4.7. schemas.py — Validación con Pydantic

```

1  from enum import StrEnum
2
3  class QuejaStatus(StrEnum):
4      PENDIENTE = "PENDIENTE"
5      EN_COLA = "EN_COLA"
6      PROCESANDO = "PROCESANDO"
7      ANALIZADA = "ANALIZADA"
8      NOTIFICADA = "NOTIFICADA"
9      ERROR = "ERROR"
10
11 class CategoriaEnum(StrEnum):
12     ACADEMICA = "ACADEMICA"
13     INFRAESTRUCTURA = "INFRAESTRUCTURA"
14     ACOSO = "ACOSO"
15     ADMINISTRATIVA = "ADMINISTRATIVA"
16     SALUD = "SALUD"
17     OTRA = "OTRA"
18
19 class QuejaCreate(QuejaBase):
20     titulo: str = Field(min_length=5, max_length=120)
21     descripcion: str = Field(min_length=20, max_length=5000)
22     categoriaDeclarada: CategoriaEnum = CategoriaEnum.OTRA
23     anonima: bool = False
24     sede: str | None = None
25     facultad: str | None = None
26     contactoEmail: EmailStr | None = None
27     contactoTelefono: str | None = Field(default=None,
28                                         pattern=r"^\+?[0-9\s\-\]{7,20}$")
29
30 class QuejaAccepted(BaseModel):
31     quejaId: str
32     status: QuejaStatus = QuejaStatus.PENDIENTE
33     correlationId: str
34     createdAt: str
35     estimatedAnalysisTime: int = 30

```

Listing 9: handlers/_shared/schemas.py (modelos principales)

Decisiones:

- **✓ Nombres de campos en camelCase** (alineados con OpenAPI)

- ✓ **Enums UPPERCASE** (alineados con spec)
- ✓ **Validaciones inline**: min_length, max_length, pattern, etc.
- ✓ **QuejaAccepted** separado de **QuejaResponse** (uno para 202, otro para 200)

4.8. auth.py — Multi-tenant

```

1 def extract_tenant_id(event):
2     headers = event.get("headers", {}) or {}
3     tenant_id = (headers.get("X-Tenant-ID")
4                 or headers.get("x-tenant-id")
5                 or headers.get("X-Tenant-Id"))
6     if not tenant_id:
7         raise UnauthorizedError(
8             "Missing X-Tenant-ID header. Multi-tenant access
9             requires..."
10        )
11    tenant_id = tenant_id.strip()
12    if not tenant_id or not all(c.isalnum() or c in "-_" for c in
13        tenant_id):
14        raise UnauthorizedError("Invalid X-Tenant-ID format")
15    return tenant_id

```

Listing 10: handlers/_shared/auth.py

Conceptos:

- ✓ Tenant ID viene de header **nunca** del body (seguridad)
- ✓ Sanitización: solo alfanumérico, guión bajo, guión
- ✗ Sin header → 401 UNAUTHORIZED

5. Las 4 Lambdas

5.1. health_check.py — Endpoint sin auth

```

1 from handlers._shared.errors import error_handler
2 from handlers._shared.http_response import HttpResponse
3 from handlers._shared.config import get_settings
4 from handlers._shared.correlation_id import with_correlation_id
5
6 @error_handler
7 def lambda_handler(event, context):
8     with_correlation_id(event)
9     settings = get_settings()
10    return HttpResponse.ok({
11        "status": "healthy",
12        "service": settings.app_name,
13        "stage": settings.stage,
14        "region": settings.region,
15    })

```

Listing 11: handlers/health_check.py

5.2. create_queja.py — POST 202 Accepted (async pattern)

```

1 @error_handler
2 def lambda_handler(event, context):
3     with_correlation_id(event)
4     try:
5         # 1. Auth: extraer tenant (multi-tenant safety)
6         tenant_id = extract_tenant_id(event)
7         user_id = extract_user_id(event)
8         log.append_keys(tenant_id=tenant_id)
9         if user_id:
10             log.append_keys(user_id=user_id)
11
12         # 2. Parse + validate body
13         body = _parse_body(event)
14         try:
15             input_data = QuejaCreate.model_validate(body)
16         except PydanticValidationError as e:
17             errors = [{"field": "...", "message": "...", "type": "..."}
18                       for err in e.errors()]
19             raise ValidationError(
20                 "Request body validation failed",
21                 details={"errors": errors}
22             ) from e
23
24         # 3. Build accepted response
25         now = now_iso()
26         correlation_id = get_correlation_id() or ""
27         accepted = QuejaAccepted(
28             status="PENDIENTE",
29             correlationId=correlation_id,
30             createdAt=now,
31             estimatedAnalysisTime=30,
32         )
33
34         # 4. Build DynamoDB item
35         item = input_data.model_dump(mode="json")
36         item["quejaId"] = accepted.quejaId
37         item["tenantId"] = tenant_id
38         item["userId"] = user_id if not input_data.anonima else None
39         item["status"] = "PENDIENTE"
40         item["createdAt"] = now
41         item["updatedAt"] = now
42         item["escalada"] = False
43         item["pk"] = queja_pk(tenant_id, accepted.quejaId)
44         item["sk"] = queja_sk(accepted.quejaId)
45         item["gsi1pk"] = f"TENANT#{tenant_id}#STATUS#PENDIENTE"
46         item["gsi1sk"] = now
47
48         # 5. Persist (con condition: evita duplicados)
49         dynamo = get_dynamo_client()
50         dynamo.put_item(item,
51                         condition_expression="attribute_not_exists(pk)")
52
53         # 6. Return 202 Accepted
54         return HttpResponse.accepted(accepted.model_dump(mode="json"))
55     finally:
56         clear_correlation_id()

```

Listing 12: handlers/create_queja.py (nucleo)

Patrones importantes:

- ✓ **202 Accepted** (no 201): procesamiento asíncrono
- ✓ `log.append_keys(tenant_id=...)`: añade contexto a todos los logs
- ✓ `condition_expression=attribute_not_exists(pk)`: previene duplicados
- ✓ `try/finally` con `clear_correlation_id()`: limpia contexto

5.3. get_queja.py y list_quejas.py

Patrones similares:

- Validan `pathParameters` (`quejaId`)
- Lllaman a `dynamo.get_queja()` o `query_by_tenant()`
- Retornan 200 con la queja o lista
- Manejan 404 con `NotFoundError`

6. SAM template — Infraestructura como código**6.1. El template completo (infra/template.yaml)**

```

1 AWSTemplateFormatVersion: "2010-09-09"
2 Transform: AWS::Serverless-2016-10-31
3
4 Parameters:
5   Stage:
6     Type: String
7     Default: dev
8     AllowedValues: [dev, staging, prod]
9   LambdaRoleArn:
10    Type: String
11    Default: "arn:aws:iam::227165337884:role/LabRole"
12
13 Globals:
14   Function:
15     Runtime: python3.12
16     MemorySize: 256
17     Timeout: 30
18     Architectures: [x86_64]
19     Tracing: Active
20     Environment:
21       Variables:
22         STAGE: !Ref Stage
23         LOG_LEVEL: INFO
24         ALLOWED_ORIGINS: "*"
25         POWERTOOLS_SERVICE_NAME: sentinel-academia
26   Api:
27     Cors:
28       AllowMethods: "'GET,POST,PUT,DELETE,OPTIONS'"
29       AllowHeaders:
30         "'Content-Type,Authorization,X-Correlation-ID,X-Tenant-ID'"

```

```

30     AllowOrigin: "'*'"
31
32 Resources:
33   QuejasTable:
34     Type: AWS::DynamoDB::Table
35     Properties:
36       TableName: !Sub "sentinel-quejas-${Stage}"
37       BillingMode: PAY_PER_REQUEST
38       AttributeDefinitions:
39         - { AttributeName: pk, AttributeType: S }
40         - { AttributeName: sk, AttributeType: S }
41         - { AttributeName: gsi1pk, AttributeType: S }
42         - { AttributeName: gsi1sk, AttributeType: S }
43       KeySchema:
44         - { AttributeName: pk, KeyType: HASH }
45         - { AttributeName: sk, KeyType: RANGE }
46       GlobalSecondaryIndexes:
47         - IndexName: GSI1-StatusByDate
48           KeySchema:
49             - { AttributeName: gsi1pk, KeyType: HASH }
50             - { AttributeName: gsi1sk, KeyType: RANGE }
51           Projection: { ProjectionType: ALL }
52
53   CreateQuejaFunction:
54     Type: AWS::Serverless::Function
55     Properties:
56       Role: !Ref LambdaRoleArn          # <-- LabRole pre-existente
57       FunctionName: !Sub "sentinel-create-queja-${Stage}"
58       CodeUri: ../src                    # <-- desde la raiz del
59         proyecto
60       Handler: handlers.create_queja.lambda_handler
61       MemorySize: 512
62     Environment:
63       Variables:
64         DYNAMODB_TABLE: !Sub "sentinel-quejas-${Stage}"
65   Events:
66     CreateApi:
67       Type: Api
68       Properties:
69         Path: /api/quejas                # <-- alineado con OpenAPI
70         Method: POST
71         RestApiId: !Ref SentinelApi

```

Listing 13: infra/template.yaml — fragmentos clave

7. AWS Academy: las trampas y cómo resolverlas

7.1. Regla de oro: SIEMPRE LabRole

✗ NO crees roles, ✓ usa LabRole

En AWS Academy Learner Lab, el usuario voclabs **NO puede**:

- iam:CreateRole — denegado
- iam:PassRole — solo sobre LabRole
- Tags en IAM resources — denegado

Consecuencia: SAM normalmente auto-genera roles. Eso falla. Solución: declarar el role manualmente.

```

1 LambdaRoleArn:
2   Type: String
3   Default: "arn:aws:iam::227165337884:role/LabRole"
4
5 CreateQuejaFunction:
6   Properties:
7     Role: !Ref LambdaRoleArn    # <-- en CADA Lambda

```

Listing 14: Cómo referenciar LabRole

7.2. No usar Políticas: en las funciones

Si pones:

```

1 Properties:
2   Policies:
3     - AWSLambdaBasicExecutionRole

```

SAM intenta crear un role nuevo → 403 AccessDenied.

Solución: no declarar Políticas; el LabRole ya tiene los permisos necesarios.

7.3. CodeUri: ../src (no ../ ni ../../)

El CodeUri determina qué se incluye en el zip de la Lambda:

```

1 # MAL: incluye TODO el proyecto (docs, web, AGENTS.md, .gitignore...)
2 CodeUri: ../
3
4 # MAL: incluye src/ + el template otra vez
5 CodeUri: ../src/../
6
7 # BIEN: solo src/, zip limpio
8 CodeUri: ../src

```

Listing 15: Comparación de CodeUri

Verificación: tras `sam build`, el zip debe tener:

- handlers/__init__.py (marcando el package)
- handlers/_shared/__init__.py
- handlers/create_queja.py
- Site-packages (boto3, pydantic, etc.)

7.4. El bug No module named 'src'

Cuando el handler era `src.handlers.create_queja.lambda_handler`, Lambda decía:

```
1 Runtime.ImportModuleError: No module named 'src'
```

Causa: aunque el zip tenía `src/`, el handler buscó `src.handlers.*` y no encontró `src` en el path de importación.

Solución: renombrar handlers para que sean **top-level**:

```
1 # Cambiar todas las referencias en el código
2 sed -i 's|from src\.handlers|from handlers|g' src/handlers/**/*.py
3 sed -i 's|src\.handlers\.|handlers.|g' template.yaml
```

8. Build, deploy, y samconfig.toml

8.1. samconfig.toml — Configuración persistente

```
1 version = 0.1
2
3 [default.deploy.parameters]
4 stack_name = "sentinel-academia-dev"
5 s3_bucket = "sentinel-sam-artifacts-1781951709"
6 s3_prefix = "sentinel-academia-dev"
7 region = "us-east-1"
8 confirm_changes_before_deploy = false
9 capabilities = "CAPABILITY_IAM"
10 disable_rollback = false
11
12 [default.deploy.global.parameters]
13 debug = false
```

Listing 16: infra/samconfig.toml

8.2. El ciclo build → deploy

```
1 # 1. Limpiar build anterior
2 rm -rf .aws-sam .aws-sam-new
3
4 # 2. Build (crea .aws-sam/build/<Function>/ con deps instaladas)
5 sam build --build-dir .aws-sam-new
6
7 # 3. Deploy (sube zips a S3 + crea/actualiza stack)
8 sam deploy --stack-name sentinel-academia-dev \
9   --template-file .aws-sam-new/template.yaml \
10  --s3-bucket sentinel-sam-artifacts-1781951709 \
11  --capabilities CAPABILITY_IAM \
12  --no-disable-rollback \
13  --no-confirm-changeset
14
15 # 4. Ver outputs
16 aws cloudformation describe-stacks \
17   --stack-name sentinel-academia-dev \
18   --query 'Stacks[0].Outputs'
```

Listing 17: Comandos esenciales

Trampa #3: __pycache__ con permisos de root

Tras `sam build` con Docker, se crean archivos `__pycache__/*.pyc` con owner `root`. No puedes borrarlos con tu usuario.

Solución: usa `--build-dir` apuntando a un directorio nuevo:

```
1 sam build --build-dir .aws-sam-new
```

9. Los 5 smoke tests que validan todo

```
1 API="https://om3eiczjr9.execute-api.us-east-1.amazonaws.com/dev"
2
3 # Test 1: Health check
4 curl -s "$API/health"
5 # Esperado: 200 + {"status":"healthy",...}
6
7 # Test 2: Crear queja (alineado con OpenAPI)
8 curl -s -X POST "$API/api/quejas" \
9   -H "Content-Type: application/json" \
10  -H "X-Tenant-ID: utpl" \
11  -H "X-User-ID: estudiante-001" \
12  -H "X-Correlation-ID: test-001" \
13  -d
14    '{"titulo":"Problema","descripcion":"...","categoriaDeclarada":"ADMINISTRATIVA"'
15 # Esperado: 202 + QuejaAccepted
16
17 # Test 3: Listar quejas
18 curl -s "$API/api/quejas" -H "X-Tenant-ID: utpl"
19 # Esperado: 200 + {items, total, nextCursor}
20
21 # Test 4: Obtener queja por ID
22 curl -s "$API/api/quejas/<id>" -H "X-Tenant-ID: utpl"
23 # Esperado: 200 + queja completa
24
25 # Test 5: Sin X-Tenant-ID (debe fallar con 401)
26 curl -s -X POST "$API/api/quejas" -H "Content-Type: application/json"
27   -d '...'
28 # Esperado: 401 +
29   {code:UNAUTHORIZED,message:...,correlationId:...,timestamp:...}
```

Listing 18: tests/smoke.sh

9.1. Output esperado de cada test

```
1 {
2   "status": "healthy",
3   "service": "sentinel-academia",
4   "stage": "dev",
5   "region": "us-east-1"
6 }
```

Listing 19: Test 1: GET /health

```
1 {
2   "quejaId": "1f6733ea-0578-48cf-af10-740962f7ced0",
3   "status": "PENDIENTE",
```



```

4  "correlationId": "test-001",
5  "createdAt": "2026-06-20T13:34:24.779106+00:00",
6  "estimatedAnalysisTime": 30
7  }

```

Listing 20: Test 2: POST /api/quejas

```

1  {
2  "code": "UNAUTHORIZED",
3  "message": "Missing X-Tenant-ID header. Multi-tenant access requires
4  a tenant identifier.",
5  "correlationId": "",
6  "timestamp": "2026-06-20T13:40:38.705363+00:00"
7  }

```

Listing 21: Test 5: error 401 (formato spec-aligned)

10. Catálogo de errores y soluciones

Error 1: iam:CreateRole denied

Cuando: sam deploy intenta crear el role de la Lambda.

Causa: AWS Academy no permite crear roles.

Solución: usar LabRole pre-existente (Role: !Ref LambdaRoleArn).

Error 2: iam:PassRole denied

Cuando: sam deploy con otro role que no sea LabRole.

Causa: el usuario voclabs solo puede pasar LabRole.

Solución: cambiar el ARN a LabRole.

Error 3: Tags denied en IAM resources

Cuando: el template tiene Tags: en una función.

Causa: Academy bloquea tagging en IAM.

Solución: quitar Tags del template.

Error 4: No module named 'src'

Cuando: handler con prefijo src.handlers.*.

Causa: el paquete src no está en el path de importación.

Solución: handlers como top-level (handlers.create_queja.lambda_handler).

Error 5: email-validator not installed

Cuando: el body usa EmailStr de Pydantic.

Causa: Pydantic no incluye email-validator por defecto.

Solución: pip install email-validator y agregar a requirements.txt.

Error 6: KeyError 'message' in LogRecord

Cuando: logger.warning(extra={'message': ...}).

Causa: message es atributo interno de logging.

Solución: usar `error_message` u otro nombre.

Error 7: Stack en ROLLBACK_COMPLETE

Cuando: primer deploy falla y no se puede re-deployar.

Causa: CloudFormation no permite update sobre ROLLBACK_COMPLETE.

Solución:

```
1 aws cloudformation delete-stack --stack-name sentinel-academia-dev
2 aws cloudformation wait stack-delete-complete --stack-name
  sentinel-academia-dev
3 sam deploy ...
```

Error 8: __pycache__ permission denied en rebuild

Cuando: `sam build` después de un build con Docker.

Causa: archivos creados por root en container Docker.

Solución: `sam build --build-dir .aws-sam-new`

11. Lo que aprendimos (resumen ejecutivo)

Checklist Fase 1

1. 1 Lambda = 1 archivo = 1 endpoint
2. `__shared/` para código común (logger, errors, http, schemas)
3. `LabRole` para todas las Lambdas (nunca crear roles)
4. `CodeUri`: `../src` apuntando a `src/`, no al project root
5. **Handlers como top-level** (sin prefijo `src.`)
6. **202 Accepted** para operaciones asíncronas
7. **Error format spec-aligned**: `{code, message, correlationId, timestamp}`
8. **Multi-tenant via X-Tenant-ID header** (nunca del body)
9. **DynamoDB single-table** con PK `TENANT#xxx#QUEJA#yyy`
10. **email-validator** obligatorio para `EmailStr`
11. **No usar 'message' en logger extra** (usar `'error_message'`)
12. **sam build --build-dir .aws-sam-new** para evitar problemas de permisos

12. Ejercicios para practicar

12.1. Ejercicio 1: Añadir un endpoint `GET /api/quejas/{id}/status`

Endpoint que devuelve solo el status actual de una queja (no toda la data).

Pista: crea `src/handlers/get_queja_status.py`, reutiliza `dynamo.get_queja()`, y mapea el schema.

12.2. Ejercicio 2: Añadir filtro por status en `list_quejas`

Modifica el handler para que acepte `?status=PENDIENTE` y use el GSI1.

Pista: cambia `query_by_tenant()` para que use el GSI cuando hay filtro de status.

12.3. Ejercicio 3: Añadir paginación con cursor real

Hoy `nextCursor: null`. Implementa cursor opaco basado en `LastEvaluatedKey` de DynamoDB.

12.4. Ejercicio 4: Crear script de smoke tests

Automatiza los 5 tests en `tests/smoke.sh` con aserciones (jq) y exit codes.

12.5. Ejercicio 5: Deploy con `sam sync`

En vez de `sam deploy`, usa `sam sync --stack-name ... --watch` para iterar más rápido.

13. Siguiente paso: Fase 2

En la **Fase 2** agregamos el LLM:

- Nuevo servicio `src/services/llm_client.py` que envuelve OCI Cohere
- Lambda `analyze_queja.py` consumer de SQS
- SQS queue para procesamiento asíncrono
- Función que toma una queja, llama Cohere, actualiza status a ANALIZADA
- El `createQueja` envía mensaje a SQS en vez de procesar directo

Tiempo estimado: 3-4 horas.

Fase	Estado
0. Setup + OCI	✓
1. Backend Core (este tutorial)	✓
2. LLM integration	próximo
3. Frontend deploy	pendiente
4. Archivos + Email	pendiente
5. RAG	opcional
6. Testing	pendiente
7. Deploy final + demo	pendiente

Fin del tutorial. qA por la Fase 2!